LL-Parser

BC George (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Wiederholung

PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie könnnen wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

Motivation

Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse erstellen lässt
- einen Algorithmus zum sog. Parsen von Programmen mit Hilfe einer solchen Grammatik

Themen für heute

- Der Einsatz kontextfreier Grammatik zur Syntaxanalyse mittels Top-Down-Techniken

Syntaxanalyse

Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

Diese vorgegebene Syntax wird im Compilerbau mit einer Grammatik beschrieben.

Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltablelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.

Arten der Syntaxanalyse

Die Syntax bezieht sich auf die Struktur der zu analysierenden Eingabe, z. B. einem Computerprogramm in einer Hochsprache. Diese Struktur wird mit formalen Grammatiken beschrieben. Einsetzbar sind Grammatiken, die deterministisch kontextfreie Sprachen erzeugen.

- Top-Down-Analyse: Aufbau des Parse trees von oben
  - Parsen durch rekursiven Abstieg
  - LL-Parsing
- (Bottom-Up-Analyse: LR-Parsing)

Mehrdeutigkeiten

Wir können nur mit eindeutigen Grammatiken arbeiten, aber:

Def.: Eine formale Sprache L heißt *inhärent mehrdeutige Sprache*, wenn jede formale Grammatik G mit $L(G) = L$ mehrdeutig ist.

Das heißt, solche Grammatiken existieren.

$\Rightarrow$ Es gibt keinen generellen Algorithmus, um Grammatiken eindeutig zu machen.

Bevor wir richtig anfangen...

Def.: Ein Nichtterminal A einer kontextfreien Grammatik G heißt *unerreichbar*, falls es kein $a, b \in (N \cup T)^*$ gibt mit $S \overset{*}{\Rightarrow} aAb$. Ein Nichtterminal A einer Grammatik G heißt *nutzlos*, wenn es kein Wort $w \in T^*$ gibt mit $A \overset{*}{\Rightarrow} w$.

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ heißt *reduziert*, wenn es keine nutzlosen oder unerreichbaren Nichtterminale in N gibt.

Bevor mit einer Grammatik weitergearbeitet wird, müssen erst alle nutzlosen und dann alle unerreichbaren Symbole eliminiert werden. Wir betrachten ab jetzt nur reduzierte Grammatiken.

Top-Down-Analyse

Wie würden Sie manuell parsen?

Algorithmus: Rekursiver Abstieg

Hier ist ein einfacher Algorithmus, der (indeterministisch) einen Ableitungsbaum vom Nonterminal X von oben nach unten aufbaut:

Eingabe: Ein Nichtterminal X und das nächste zu verarbeitende Eingabezeichen a.

```
RecursiveDescent(X, a) //X ∈ N, a ∈ T
    for a production, X → Y_1 Y_2 ... Y_n
        for i = 1 to n
            if Y_i ∈ N
                RecursiveDescent(Y_i, a)
            else if Y_i ≠ a
                error
```

Abbildung 1: Recursive Descent-Algorithmus

Grenzen des Algorithmus

Was ist mit

1) $X \to a\alpha \mid b\beta$
2) $X \to B\alpha \mid C\beta$
3) $X \to B\alpha \mid B\beta$
4) $X \to B\alpha \mid C\beta$ und $C \to B$
5) $X \to X\beta$
6) $X \to B\alpha$ und $B \to X\beta$

$X, B, C, D \in N^*; a, b, c, d \in T^*; \beta, \alpha, \beta \in (N \cup T)^*$

Linksfaktorisierung

$X \to BC \mid BD$

Algorithmus: Linksfaktorisierung

Eingabe: Eine Grammatik G = (N, T, P, S)

Ausgabe: Eine äquivalente links-faktorisierte Grammatik G'

```
repeat
   for each X ∈ N
      find the longest prefix α ∈ (N ∪ T)* common to at least two
         of its productions

      if α ≠ ε exists
         replace all X-productions

               X → α β1 | α β2 | ... | α βn | γ
               (with γ ≠ αδ for a δ ∈ (N ∪ T)*)

                 by the following rules with a new nonterminal Y:

                   X → α Y | γ
                   Y → β1 | β2 | ... | βn

  until there are no two X ∈ N with productions with common prefixes
```

Abbildung 2: Algorithmus zur Linksfaktorisierung

Linksrekursion

Def.: Eine Grammatik $G = (N, T, P, S)$ heißt *linksrekursiv*, wenn sie ein Nichtterminal X hat, für das es eine Ableitung $X \overset{+}{\Rightarrow} X\ \alpha$ für ein $\alpha \in (N \cup T)^*$ gibt.

Linksrekursion gibt es

direkt: $X \to X\alpha$

und

indirekt: $X \to \ldots \to \ldots \to X\alpha$

Algorithmus: Entfernung von direkter Linksrekursion

Eingabe: Eine Grammatik G = (N, T, P, S)

Ausgabe: Eine äquivalente Grammatik G' ohne direkte Linksrekursion

```
replace each production X → Xα | β by
        X → β Y
        Y → α Y | ε
```

Abbildung 3: Algorithmus zur Entfernung direkter Linksrekursion

Algorithmus: Entfernung von indirekter Linksrekursion

Eingabe: Eine Grammatik G = (N, T, P, S) mit $N = \{X_1, X_2, \ldots X_n\}$ ohne ϵ-Regeln oder Zyklen der Form $X_1 \to X_2, X_2 \to X_3, \ldots X_{m-1} \to X_m, X_m \to X_1$

Ausgabe: Eine äquivalente Grammatik G' ohne Linksrekursion

```
for i = 1 to n
   for j = 1 to i-1

      replace each X → X_j α by the productions

         X_i → β_1 α | β_2 α | ... | β_k α
         with X_j → β_1 | β_2 | ... | β_k are all the
            X_j-productions

      remove all X_i - productions with direct left recursion
```

Abbildung 4: Algorithmus zur Entfernung indirekter Linksrekursion

Arbeiten mit generierten Parsern: LL(k)-Grammatiken

First-Mengen

$S \to A \mid B \mid C$

Welche Produktion nehmen?

Wir brauchen die "terminalen k-Anfänge" von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w, \text{mit } w \in T^*, First_k(w) = \{v\}\}$

Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

Follow-Mengen

Manchmal müssen wir wissen, welche terminalen Zeichen hinter einem Nichtterminal stehen können.

Def. Wir definieren *Follow* - Mengen einer Grammatik wie folgt:

$\forall \beta \in (N \cup T)^* :$

$$Follow_k(\beta) = \{w \in T^* \mid \exists \alpha, \gamma \in (N \cup T)^* \ \text{ mit } \ S \overset{*}{\Rightarrow}_l \alpha\beta\gamma \ \text{ und } \ w \in First_k(\gamma)\}$$

LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine $LL(k)$-Grammatik, wenn für alle Linksableitungen der Form:

$S \overset{*}{\Rightarrow}_l \; wA\gamma \Rightarrow_l w\alpha\gamma \overset{*}{\Rightarrow}_l wx$

und

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\beta\gamma \overset{*}{\Rightarrow}_l wy$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$\alpha = \beta$

LL(1)-Grammatiken

LL(k)-Sprachen

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren, deren Eingabe eine LL(k)- (meist LL(1)-) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik G = (N, T, P, S)

Ausgabe: Eine Parsertabelle *P*

```
for each production X → α
   for each a ∈ First(α)
      add X → α to P[X, a]

   if ϵ ∈ First(α)
      for each b ∈ Follow(α)
         add X → α to P[X, b]
         if ϵ ∈ First(α) and ⊥ ∈ Follow(X)
            add X → α to P[A, ⊥]
```

Abbildung 5: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist $\perp$ das Endezeichen des Inputs. Statt $\mathit{First}_1(\alpha)$ und $\mathit{Follow}_1(\alpha)$ wird oft nur $\mathit{First}(\alpha)$ und $\mathit{Follow}(\alpha)$ geschrieben.

LL-Parsertabellen

LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt (bei einem Recursive-Descent-Parser, aber auch bei der Parsertabelle). Diesen Stack kann man auch "selbst programmieren", d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik G = (N, T, P, S), eine Parsertabelle P mit $w \perp$ als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w, Fehler sonst

```
a = next_token()
X = top of stack  // entfernt X vom Stack

while X ≠⊥

   if X = a
      a = next_token()

   else if X ∈ T
      error

   else if P[X,a] leer
      error

   else if P[X,a] = X → Y1Y2...Yk
      process_production(X → Y1Y2...Yk)
      push(Y1Y2...Yk) //Y1 = top of stack

   X = top of stack
```

Abbildung 6: Algorithmus zum tabellengesteuerten LL-Parsen

Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
  - Syntaxbaum oder Parse Tree
  - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, ...)
- Symboltabelle

Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein einfacher Recursive-Descent-Parser arbeitet mit Backtracking.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

LICENSE

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.